ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN A1

CO5085 · Học sâu và ứng dụng trong thị giác máy tính

Học kỳ 261 · Năm học 2026–2027

So sánh ResNet18 và Swin-T trên CIFAR-100

Học viên

Huỳnh Thành Đạt	2570161
Giảng viên	Lê Thành Sách
	Nguyễn Quốc Minh

Thành phố Hồ Chí Minh, tháng 11 năm 2026

Tóm tắt

Báo cáo so sánh ResNet18 và Swin-T trên CIFAR-100, với hai câu hỏi: trọng số pretrained từ ImageNet có giúp phân loại tốt hơn không, và nên cho phép mô hình cập nhật bao nhiêu tầng khi chuyển sang dữ liệu mới? Mỗi kiến trúc được huấn luyện từ đầu và thử ba chế độ pretrained: chỉ học head, học stage cuối và học toàn bộ mô hình. Thí nghiệm chính dùng 20 epoch và ba seed cho mỗi cấu hình.

Full fine-tuning cho kết quả tốt nhất ở cả hai kiến trúc. Swin-T đạt $86{,}54 \pm 0{,}24\%$, cao hơn $78{,}93 \pm 0{,}10\%$ của ResNet18, nhưng cần nhiều thời gian và bộ nhớ hơn. Swin-T học từ đầu gặp khó trong lịch 20 epoch. Khi dùng một lịch chung 40 epoch, bản scratch đạt $60{,}00 \pm 0{,}47\%$, còn bản pretrained đạt $86{,}45 \pm 0{,}19\%$. Kết quả này cho thấy lợi ích của pretraining vẫn lớn khi bản scratch đã học được cách phân loại.

Tổng quan báo cáo

Bài toán	Phân loại 100 lớp ảnh CIFAR-100.
Kiến trúc	ResNet18 và Swin-T. Các bản pretrained dùng trọng số ImageNet.
Đối chứng	Huấn luyện từ đầu, head-only, partial fine-tuning và full fine-tuning.
Phép đo	Độ chính xác, macro-F1, thời gian và bộ nhớ trong hai thiết lập với tổng cộng 30 lượt.
Kết quả chính	Swin-T scratch đạt 60,00% trong lịch 40 epoch. Bản pretrained cùng lịch đạt 86,45%.
Giới hạn	Ba seed trên một tập dữ liệu; đối chứng 40 epoch dành riêng cho Swin-T.

Mục lục

Tóm tắt **i**

Tổng quan báo cáo **ii**

1 Bài toán và dữ liệu **1**

1.1 Mục tiêu và phạm vi 1

1.2 Nguồn và cấu trúc CIFAR-100 1

1.3 Phân bố dữ liệu và cách chia tập 1

1.4 Tiền xử lý dữ liệu 3

1.5 Tải dữ liệu theo batch 3

2 Kiến trúc và chế độ huấn luyện **4**

2.1 ResNet18: đối chứng CNN 4

2.2 Swin-T: đối chứng Transformer 4

2.3 Ma trận đối chứng 5

3 Thiết lập thực nghiệm và phép đo **6**

3.1 Thiết lập 20 epoch 6

3.2 Thiết lập 40 epoch cho Swin-T 7

3.3 Chọn checkpoint và đánh giá trên tập test 7

3.4 Chi phí và môi trường 8

4 Kết quả trong thiết lập 20 epoch **9**

4.1 So sánh các chế độ huấn luyện 9

4.2 Thời gian, bộ nhớ và số tham số 9

4.3 Thảo luận kết quả 10

4.3.1 Ảnh hưởng của chế độ fine-tuning 10

4.3.2 Swin-T scratch trong lịch 20 epoch 10

4.3.3 Độ chính xác và chi phí tính toán 11

4.3.4 Khoảng cách train/validation và lỗi theo lớp 11
4.4 Đường cong và lỗi phân loại 12

5 Kết quả đối chứng khởi tạo Swin-T 17

6 Kết luận và giới hạn 19
6.1 Kết luận 19
6.2 Giới hạn 19

Tài liệu tham khảo 20

Danh sách hình vẽ

1.1 Số ảnh train và validation của 100 lớp CIFAR-100 sau khi chia với seed 2026: mỗi lớp có 450 ảnh train và 50 ảnh validation. 2

1.2 Mười ảnh train CIFAR-100 ở kích thước gốc 32×32, chọn ngẫu nhiên với seed 42; nhãn lớp hiển thị phía trên từng ảnh. 2

1.3 Năm ảnh train trước (hàng trên) và sau augmentation (hàng dưới), với seed 42. Random crop và horizontal flip thay đổi vị trí, hướng của vật thể nhưng giữ nhãn; đầu vào được resize từ 32×32 lên 96×96. Ảnh được đưa về thang RGB $[0, 1]$ để hiển thị. 3

4.1 Cross-entropy và accuracy trên train/validation của ResNet18 scratch ở seed 42. 12

4.2 Cross-entropy và accuracy trên train/validation của Swin-T scratch ở seed 42. 12

4.3 Cross-entropy và accuracy trên train/validation của ResNet18 pretrained full fine-tuning ở seed 42. 13

4.4 Cross-entropy và accuracy trên train/validation của Swin-T pretrained full fine-tuning ở seed 42. 13

4.5 Confusion matrix test của ResNet18 full ở seed 42; hàng là nhãn thực tế. 14

4.6 Confusion matrix test của Swin-T full ở seed 42. 15

4.7 Các dự đoán sai có confidence cao của ResNet18 full ở seed 42, hiển thị ảnh gốc 32×32. 15

4.8 Các dự đoán sai có confidence cao của Swin-T full ở seed 42. 16

5.1 Cross-entropy và accuracy trên train/validation của Swin-T scratch trong lịch 40 epoch ở seed 42. 18

5.2 Cross-entropy và accuracy trên train/validation của Swin-T pretrained full fine-tuning trong lịch 40 epoch ở seed 42. 18

Danh sách bảng

2.1 Bốn chế độ huấn luyện của ResNet18 và Swin-T trên cùng split CIFAR-100, với seed 42, 43 và 44. 5

3.1 Tham số của thí nghiệm chính trên CIFAR-100. 6

4.1 Accuracy và macro-F1 test của tám cấu hình, trung bình và SD mẫu qua seed 42, 43 và 44. Accuracy theo phần trăm, F1 theo thang 0–1. 9

4.2 Tổng tham số và tham số được học theo triệu (M). Thời gian huấn luyện 20 epoch và bộ nhớ cấp phát cực đại trên RTX 5090 là trung bình và SD mẫu qua seed 42, 43 và 44. 9

4.3 Ba lớp có recall thấp nhất của mỗi bản full fine-tuning, tính từ confusion matrix test gộp seed 42, 43 và 44; support 300 dự đoán mỗi lớp. 11

5.1 Accuracy validation/test và macro-F1 test của Swin-T scratch và pre-trained full fine-tuning dưới cùng lịch 40 epoch, trung bình và SD mẫu qua seed 42, 43 và 44. 17

5.2 Chi phí lịch 40 epoch; thời gian train và peak allocated memory là trung bình và SD mẫu qua ba seed. 17

Chương 1

Bài toán và dữ liệu

1.1 Mục tiêu và phạm vi

ResNet18 đại diện cho CNN, còn Swin-T đại diện cho Transformer. Bài toán là nhận dạng 100 lớp ảnh CIFAR-100. Với mỗi kiến trúc, báo cáo so sánh huấn luyện từ đầu với việc dùng trọng số đã học trên ImageNet. Backbone là phần biến ảnh thành các đặc trưng; head dùng các đặc trưng đó để dự đoán lớp. Ba chế độ fine-tuning cho phép lần lượt head, stage cuối hoặc toàn bộ mô hình được học trên CIFAR-100.

Thí nghiệm chính gồm tám cấu hình, mỗi cấu hình chạy 20 epoch với ba seed 42, 43 và 44, tổng cộng 24 lượt. Sáu lượt bổ sung so sánh Swin-T khởi tạo ngẫu nhiên với Swin-T pretrained dưới cùng lịch 40 epoch. Accuracy và macro-F1 đánh giá chất lượng phân loại; thời gian và bộ nhớ cho biết chi phí của từng chế độ.

1.2 Nguồn và cấu trúc CIFAR-100

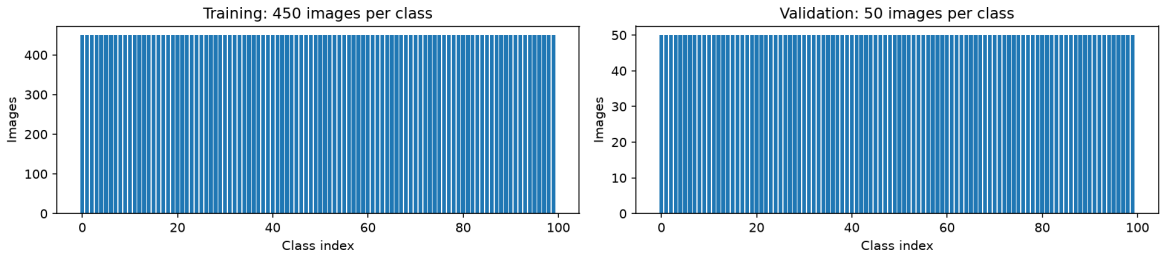
CIFAR-100 gồm 60.000 ảnh màu 32×32, thuộc 100 lớp chi tiết được xếp thành 20 nhóm tổng quát. Thực nghiệm dự đoán toàn bộ 100 lớp chi tiết. Mỗi lớp có 500 ảnh trong tập train chính thức và 100 ảnh trong tập test chính thức [2]. Số lớp lớn hơn bài E1 và ảnh độ phân giải thấp có thể làm những lớp gần nhau khó phân biệt.

Dữ liệu được cung cấp trên trang CIFAR của University of Toronto [2]. Dataset card `uoft-cs/cifar100` ghi license là *unknown* [3].

Bản Python của CIFAR-100 được phân phối trong tệp nén `cifar-100-python.tar.gz`. Sau giải nén, thư mục dữ liệu chứa ba tệp pickle: `train` và `test` lưu pixel ảnh cùng nhãn lớp chi tiết và nhãn nhóm; `meta` lưu tên các lớp. Mỗi ảnh màu có 3.072 giá trị pixel, được đọc thành mảng $32 \times 32 \times 3$ bằng `torchvision.datasets.CIFAR100`.

1.3 Phân bố dữ liệu và cách chia tập

Từ 50.000 ảnh train chính thức, lấy ngẫu nhiên 50 ảnh mỗi lớp làm validation với seed 2026. Phần còn lại dùng để huấn luyện. Ba tập có 45.000 ảnh train, 5.000 ảnh validation và 10.000 ảnh test; mỗi lớp lần lượt có 450, 50 và 100 ảnh. Cách chia được giữ nguyên cho mọi mô hình và seed.



Hình 1.1: Số ảnh train và validation của 100 lớp CIFAR-100 sau khi chia với seed 2026: mỗi lớp có 450 ảnh train và 50 ảnh validation.

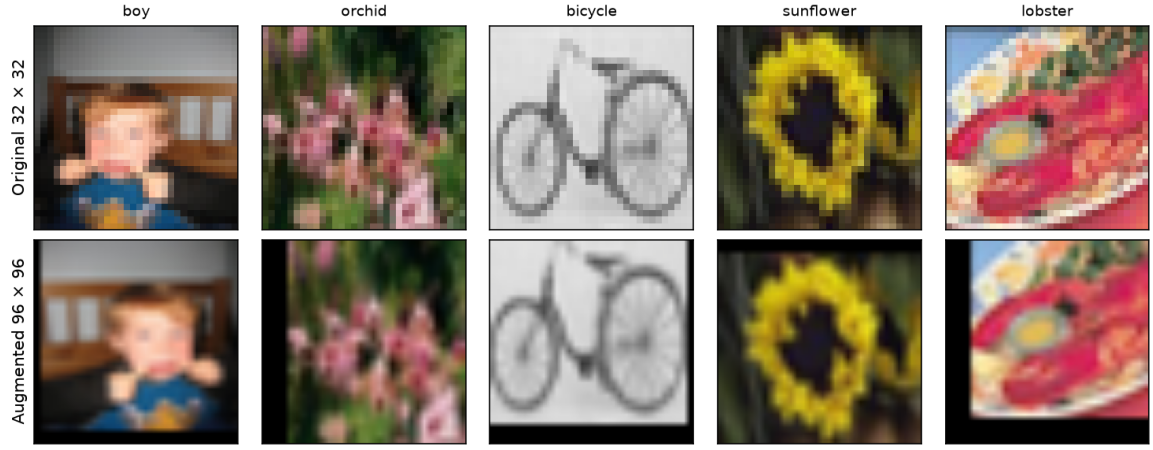


Hình 1.2: Mười ảnh train CIFAR-100 ở kích thước gốc 32×32, chọn ngẫu nhiên với seed 42; nhãn lớp hiển thị phía trên từng ảnh.

1.4 Tiền xử lý dữ liệu

Khi huấn luyện, ảnh được padding bốn pixel, cắt ngẫu nhiên về 32×32 và lật ngang với xác suất 0,5. Sau đó, ảnh được nội suy bilinear lên 96×96 với antialias. Validation và test chỉ được resize, không áp dụng biến đổi ngẫu nhiên. Đầu vào 96×96 được dùng cho cả hai kiến trúc để giảm chi phí so với 224×224; phép nội suy không tạo thêm chi tiết so với ảnh gốc.

Cả hai kiến trúc dùng thống kê ImageNet để chuẩn hóa từng kênh: trung bình $(0{,}485, 0{,}456, 0{,}406)$ và độ lệch chuẩn $(0{,}229, 0{,}224, 0{,}225)$. Trong tập train, thứ tự ảnh được xáo ở mỗi epoch. Seed của từng lượt được dùng cho khởi tạo mô hình, xáo ảnh và augmentation.



Hình 1.3: Năm ảnh train trước (hàng trên) và sau augmentation (hàng dưới), với seed 42. Random crop và horizontal flip thay đổi vị trí, hướng của vật thể nhưng giữ nhãn; đầu vào được resize từ 32×32 lên 96×96. Ảnh được đưa về thang RGB $[0, 1]$ để hiển thị.

1.5 Tải dữ liệu theo batch

Mỗi lần đọc một ảnh train, Dataset áp dụng chuỗi augmentation rồi chuyển ảnh thành tensor đã chuẩn hóa. DataLoader ghép các tensor thành batch 128 ảnh và chỉ xáo thứ tự ở tập train. Validation và test dùng phép biến đổi cố định, giữ thứ tự ảnh để đối chiếu dự đoán. Cả ba DataLoader đều giữ batch cuối nếu chưa đủ 128 ảnh (`drop_last=False`), nên mỗi epoch vẫn dùng đủ số ảnh của từng tập. Lượt chạy trên RTX 5090 dùng bốn worker tải dữ liệu; từng batch sau đó được đưa lên GPU để huấn luyện hoặc đánh giá.

Chương 2

Kiến trúc và chế độ huấn luyện

2.1 ResNet18: đối chứng CNN

Giữ kiến trúc ResNet18 chuẩn torchvision, gồm stem convolution 7×7, max pooling và bốn residual stages. Một residual block cập nhật

$$h_{l+1} = \mathrm{ReLU}\left(F_l(h_l) + S_l(h_l)\right), \tag{2.1}$$

trong đó h_l là đầu vào, $F_l(h_l)$ là kết quả của nhánh convolution và $S_l(h_l)$ là nhánh shortcut. Khi kích thước không đổi, shortcut truyền trực tiếp h_l; khi đổi số kênh hoặc stride, một phép projection đưa hai nhánh về cùng kích thước để cộng được với nhau. Khối chỉ cần học phần hiệu chỉnh $F_l(h_l)$ thay vì tạo lại toàn bộ biểu diễn. Nhánh shortcut cũng tạo đường truyền gradient qua phép cộng, giúp huấn luyện mạng sâu [4]. Sau adaptive average pooling, head Linear được thay để dự đoán 100 lớp. Stem vẫn giữ cấu trúc ResNet18 chuẩn.

Bản pretrained dùng trọng số ImageNet-1K V1 của torchvision. Ba chế độ là chỉ học head phân loại; học residual stage cuối và head; học toàn bộ. Bản scratch khởi tạo mới và học toàn bộ. BatchNorm trong phần frozen ở eval, nên cả tham số lẫn running mean/variance và bộ đếm batch được giữ nguyên.

2.2 Swin-T: đối chứng Transformer

Swin-T biến các vùng ảnh nhỏ thành token bằng patch embedding stride 4. Các token trao đổi thông tin qua attention trong từng cửa sổ. Khác với convolution có kernel dùng chung theo vị trí, trọng số attention được tính từ nội dung các token đang xét. Window attention có dạng

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} + B + M\right)V, \tag{2.2}$$

Trong một attention head, Q, K và V được tạo bằng ba phép chiếu tuyến tính từ các token. Tích $QK^\top$ đo mức liên quan giữa các cặp token; chia cho $\sqrt{d}$ giúp kiểm soát độ lớn khi số chiều query/key là d. Softmax biến các điểm này thành trọng số, rồi lấy tổng có trọng số của V để tạo biểu diễn mới. B bổ sung thông tin vị trí tương đối; M chặn

những cặp token không được phép tương tác. Các head được ghép lại rồi chiếu tuyến tính.

Swin luân phiên cửa sổ thường và cửa sổ dịch chuyển. Nhờ vậy, những token ở hai phía ranh giới cửa sổ vẫn có thể trao đổi thông tin qua các block liên tiếp. Patch merging ghép các vùng lân cận, làm lưới token nhỏ dần và tăng số kênh đặc trưng ở các stage sâu [5].

Swin-T giữ kiến trúc chuẩn torchvision, với head phân loại được thay bằng Linear 100 lớp. Đầu vào 96×96 tạo lưới patch đầu 24×24; các stage muộn có lưới nhỏ hơn. Kích thước cửa sổ mặc định là 7, với padding nội bộ ở những stage có kích thước không chia hết cho cửa sổ.

Bản pretrained dùng trọng số ImageNet-1K V1 của torchvision. Chế độ partial chỉ học stage cuối, LayerNorm cuối và head; patch merging đứng trước stage cuối vẫn frozen. Head-only chỉ học head. Frozen stages được giữ ở eval, bao gồm stochastic depth, để đặc trưng của phần đóng băng ổn định trong quá trình học head. Full và scratch cập nhật toàn bộ mô hình.

2.3 Ma trận đối chứng

Chế độ	Khởi tạo backbone	Phần được cập nhật
Scratch	Ngẫu nhiên	Toàn bộ
Head-only	ImageNet pretrained	Head 100 lớp
Partial	ImageNet pretrained	Stage cuối và head; Swin thêm norm
Full	ImageNet pretrained	Toàn bộ

Bảng 2.1: Bốn chế độ huấn luyện của ResNet18 và Swin-T trên cùng split CIFAR-100, với seed 42, 43 và 44.

Head-only trả lời liệu đặc trưng ImageNet có thể dùng ngay cho CIFAR-100 khi chỉ học bộ phân loại mới. Partial fine-tuning cho stage cuối thích nghi với dữ liệu, còn full fine-tuning cho phép cả các tầng trước đó thay đổi. Scratch là baseline học toàn bộ mô hình từ trọng số ngẫu nhiên. So sánh bốn chế độ giúp thấy vai trò của cả điểm khởi đầu và mức độ thích nghi của backbone.

Chương 3

Thiết lập thực nghiệm và phép đo

3.1 Thiết lập 20 epoch

Tám cấu hình dùng cùng số epoch, batch size và cách chia dữ liệu. Scratch và head-only dùng learning rate lớn hơn để học các trọng số mới. Partial và full dùng learning rate nhỏ hơn khi điều chỉnh backbone pretrained. Bảng sau tóm tắt các giá trị được dùng.

Tham số	Giá trị
Số epoch và batch size	20 epoch; 128 ảnh/batch; giữ batch cuối
Seed huấn luyện	42, 43 và 44
Optimizer	AdamW; $\beta = (0{,}9, 0{,}999)$; $\epsilon = 10^{-8}$
Weight decay	10^{-4}
Learning rate ban đầu	10^{-3} cho scratch/head-only; 10^{-4} cho partial/full
Lịch learning rate	Giảm theo cosine về 0, cập nhật sau mỗi epoch
Gradient clipping	Giới hạn chuẩn gradient toàn mô hình ở 1
Kiểu tính toán	BF16 autocast; trọng số và trạng thái AdamW giữ FP32

Bảng 3.1: Tham số của thí nghiệm chính trên CIFAR-100.

Learning rate sau e epoch được tính theo

$$\eta_e = \frac{\eta_0}{2}\left[1 + \cos\left(\frac{\pi e}{20}\right)\right], \tag{3.1}$$

với $e = 0$ trước epoch đầu. Loss cross-entropy không label smoothing:

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp z_{i,y_i}}{\sum_{c=1}^{100} \exp z_{i,c}}. \tag{3.2}$$

Ở đây, B là số ảnh trong batch, $z_{i,c}$ là logit của ảnh i tại lớp c và y_i là nhãn đúng. Softmax chuyển logits thành xác suất; cross-entropy nhỏ khi mô hình gán xác suất cao cho nhãn đúng. Mỗi batch được đưa qua mô hình để tính loss, sau đó backward tính gradient và AdamW cập nhật các tham số được phép học. Các bước này được viết trực tiếp bằng PyTorch.

Thí nghiệm chạy trên RTX 5090 với PyTorch 2.10.0, CUDA 13.0 và torchvision 0.25.0. DataLoader dùng bốn worker; CPU dùng bốn luồng. Mọi cấu hình dùng BF16 autocast, tắt TF32 và không dùng compile.

3.2 Thiết lập 40 epoch cho Swin-T

Trong lịch 20 epoch, Swin-T scratch học kém ngay trên train. Vì vậy, khoảng cách với bản pretrained vừa phản ánh lợi ích của trọng số ImageNet, vừa phản ánh việc bản scratch chưa được tối ưu hiệu quả. Lịch 40 epoch được dùng để kiểm tra lại hai cách khởi tạo khi bản scratch học được tốt hơn. Trong đối chứng 40 epoch, hai bản Swin-T cùng cập nhật toàn bộ tham số và chỉ khác nguồn trọng số ban đầu. Kiến trúc, cách chia dữ liệu, tiền xử lý, batch size và ba seed được giữ nguyên. Cả hai dùng AdamW với weight decay 10^{-2}. Learning rate tăng tuyến tính từ 2×10^{-5} lên 2×10^{-4} trong năm epoch đầu, rồi giảm theo cosine về 0 trong 35 epoch còn lại.

Warmup tăng learning rate dần ở đầu quá trình học; cosine giảm learning rate về cuối để các bước cập nhật nhỏ hơn. Cách kết hợp này được tham khảo từ bài báo Swin [5], với learning rate và số epoch điều chỉnh cho CIFAR-100. Hai cách khởi tạo được so sánh trong cùng lịch 40 epoch.

3.3 Chọn checkpoint và đánh giá trên tập test

Cuối mỗi epoch, mô hình được đánh giá trên validation. Checkpoint có accuracy cao nhất được giữ lại; nếu bằng nhau, chọn checkpoint có cross-entropy thấp hơn, rồi ưu tiên epoch sớm hơn. Validation chỉ phục vụ lựa chọn checkpoint, không dùng để cập nhật trọng số.

Sau khi hoàn tất huấn luyện và cố định các cấu hình, checkpoint đã chọn được đánh giá trên tập test. Tập test không được dùng để điều chỉnh learning rate, chọn seed hoặc thay đổi mô hình.

Accuracy và macro-F1 đều tính trên 100 lớp. Với mỗi lớp c,

$$F1_c = \frac{2TP_c}{2TP_c + FP_c + FN_c}, \qquad F1_{macro} = \frac{1}{100}\sum_{c=1}^{100} F1_c. \tag{3.3}$$

Với lớp c, TP_c đếm ảnh của lớp được dự đoán đúng, FP_c đếm ảnh lớp khác bị gán vào c, còn FN_c đếm ảnh lớp c bị bỏ sót. F1 kết hợp precision và recall; macro-F1 lấy trung bình F1 của 100 lớp nên mỗi lớp có trọng số như nhau. Nếu mô hình dồn nhiều ảnh vào một vài lớp, macro-F1 sẽ thấp vì các lớp còn lại bị bỏ sót. Mẫu số bằng 0 được quy ước có F1 bằng 0.

Confusion matrix cho biết mỗi lớp thường bị nhầm sang lớp nào. Các bảng theo lớp bổ sung support, precision, recall và F1. Gallery dùng tối đa tám ảnh dự đoán sai có confidence cao nhất ở seed 42 để xem những lỗi mô hình vẫn trả lời rất chắc chắn.

3.4 Chi phí và môi trường

Thời gian huấn luyện bao gồm tải ảnh, augmentation và cập nhật trọng số. Thời gian mỗi epoch bao gồm thêm bước đánh giá validation. GPU được đồng bộ trước và sau khi đo; thời gian dựng mô hình, lưu checkpoint và xuất báo cáo không nằm trong các khoảng này.

Bộ nhớ GPU được báo cáo là lượng bộ nhớ tensor được cấp phát cực đại trong một lượt huấn luyện và validation. Chỉ số này không bao gồm toàn bộ VRAM mà hệ thống giữ chỗ. Các phép đo dùng cùng môi trường phần cứng và phần mềm để so sánh chi phí giữa các chế độ.

Notebook A1 trình bày từ khảo sát dữ liệu đến huấn luyện và đánh giá. Notebook kèm cấu hình và kết quả theo từng seed.

Chương 4

Kết quả trong thiết lập 20 epoch

4.1 So sánh các chế độ huấn luyện

Cấu hình	Validation (%)	Test (%)	Macro-F1
ResNet18 Scratch	64,25 ± 0,18	64,53 ± 0,36	0,6441 ± 0,0032
ResNet18 Head-only	62,06 ± 0,12	60,89 ± 0,06	0,6076 ± 0,0008
ResNet18 Partial	74,04 ± 0,15	73,91 ± 0,18	0,7388 ± 0,0017
ResNet18 Full	78,87 ± 0,19	78,93 ± 0,10	0,7891 ± 0,0011
Swin-T Scratch	2,26 ± 1,49	2,19 ± 1,35	0,0067 ± 0,0075
Swin-T Head-only	72,37 ± 0,11	71,75 ± 0,05	0,7161 ± 0,0007
Swin-T Partial	78,69 ± 0,06	78,34 ± 0,26	0,7826 ± 0,0026
Swin-T Full	86,98 ± 0,11	86,54 ± 0,24	0,8653 ± 0,0025

Bảng 4.1: Accuracy và macro-F1 test của tám cấu hình, trung bình và SD mẫu qua seed 42, 43 và 44. Accuracy theo phần trăm, F1 theo thang 0–1.

Kết quả được tổng hợp bằng trung bình và độ lệch chuẩn mẫu qua ba seed.

4.2 Thời gian, bộ nhớ và số tham số

Kiến trúc	Chế độ	Tổng (M)	Học (M)	Giây	MiB
ResNet18	Scratch	11,228	11,228	76,4 ± 3,3	522,5 ± 0,1
ResNet18	Head-only	11,228	0,051	64,0 ± 0,1	252,1 ± 0,0
ResNet18	Partial	11,228	8,445	64,4 ± 0,3	347,3 ± 0,0
ResNet18	Full	11,228	11,228	66,8 ± 0,1	522,6 ± 0,0
Swin-T	Scratch	27,596	27,596	213,0 ± 0,5	2456,9 ± 0,0
Swin-T	Head-only	27,596	0,077	85,0 ± 0,1	502,2 ± 0,0
Swin-T	Partial	27,596	14,262	106,3 ± 0,3	663,6 ± 0,0
Swin-T	Full	27,596	27,596	214,1 ± 0,3	2456,9 ± 0,0

Bảng 4.2: Tổng tham số và tham số được học theo triệu (M). Thời gian huấn luyện 20 epoch và bộ nhớ cấp phát cực đại trên RTX 5090 là trung bình và SD mẫu qua seed 42, 43 và 44.

4.3 Thảo luận kết quả

4.3.1 Ảnh hưởng của chế độ fine-tuning

Ở cả hai kiến trúc, accuracy tăng khi cho phép nhiều tầng pretrained được cập nhật hơn. Với ResNet18, head-only đạt 60,89%, còn full fine-tuning đạt 78,93%. Với Swin-T, hai mức tương ứng là 71,75% và 86,54%. Partial fine-tuning nằm giữa hai chế độ này.

Head-only giữ nguyên cách backbone mô tả ảnh và chỉ học bộ phân loại trên các đặc trưng đó. Khi chuyển sang partial hoặc full fine-tuning, mô hình còn được thay đổi các đặc trưng để phù hợp với CIFAR-100. Kết quả cho thấy bước thích nghi này có ích ở cả CNN và Transformer. Đặc biệt, Swin-T tăng thêm 8,20 điểm phần trăm khi chuyển từ partial sang full: cập nhật các stage trước cũng đem lại lợi ích, dù stage cuối đã được học ở chế độ partial.

ResNet18 scratch đạt 64,53%, cao hơn head-only nhưng thấp hơn partial và full. Như vậy, dùng trọng số pretrained không tự bảo đảm kết quả tốt hơn nếu backbone bị giữ nguyên. Mức lợi ích phụ thuộc vào cả biểu diễn ban đầu và cách cho phép mô hình thích nghi với dữ liệu mới.

4.3.2 Swin-T scratch trong lịch 20 epoch

Swin-T scratch chỉ đạt accuracy test trung bình 2,19% trong lịch 20 epoch. Điểm thấp không chỉ xuất hiện trên test: ở cuối lịch, hai seed có accuracy train dưới 1% và cross-entropy gần $\ln(100) \approx 4{,}61$. Đây là mức loss của dự đoán gần đều trên 100 lớp. Mô hình chưa học tốt ngay cả dữ liệu dùng để cập nhật trọng số, nên vấn đề chính ở các lượt này là quá trình học chưa hiệu quả.

Confusion matrix giúp thấy rõ hành vi đó. Ở checkpoint của seed 42, mô hình dự đoán mọi ảnh test vào cùng một lớp. Vì test có 100 ảnh cho mỗi lớp, accuracy vẫn bằng 1%, nhưng F1 của phần lớn lớp bằng 0. Điều này giải thích vì sao macro-F1 trung bình của cấu hình chỉ đạt 0,0067, thấp hơn accuracy 0,0219 khi cùng viết trên thang 0–1.

ResNet18 dùng convolution để kết hợp thông tin cục bộ ngay từ những tầng đầu. Swin-T học cách kết hợp các patch bằng attention, còn bản pretrained đã có các trọng số học từ ImageNet. Trong lịch 20 epoch đang xét, hai cách khởi tạo Swin-T vì vậy bắt đầu ở mức biểu diễn rất khác nhau. Đối chứng 40 epoch kiểm tra lợi ích của pretraining trong điều kiện bản scratch đã học được, thay vì chỉ dựa vào khoảng cách với một cấu hình gần mức đoán ngẫu nhiên.

4.3.3 Độ chính xác và chi phí tính toán

Ở chế độ full fine-tuning, Swin-T cao hơn ResNet18 trung bình $7{,}61 \pm 0{,}17$ điểm phần trăm khi so sánh các cặp cùng seed. Swin-T tốt hơn ở cả ba cặp. Đổi lại, thời gian train dài gấp 3,21 lần và bộ nhớ tensor cấp phát cực đại lớn gấp 4,70 lần. Vì vậy, Swin-T full là lựa chọn tốt hơn về accuracy trong thí nghiệm này, còn ResNet18 full tiết kiệm tài nguyên hơn.

Đóng băng các tầng giúp giảm chi phí backward và lưu gradient, nhưng mô hình vẫn phải forward qua backbone để tạo đặc trưng. Điều này giải thích vì sao số tham số trainable giảm mạnh mà thời gian có thể giảm ít. Với ResNet18, ba chế độ pretrained đều mất khoảng 64–67 giây train. Với Swin-T, partial chỉ mất khoảng một nửa thời gian của full, nhưng accuracy thấp hơn 8,20 điểm phần trăm.

Stage cuối chứa nhiều kênh đặc trưng, nên partial vẫn cập nhật một tỷ lệ đáng kể tham số: khoảng ba phần tư mô hình ResNet18 và hơn một nửa Swin-T. Bảng chi phí cho thấy rõ sự khác nhau giữa số tham số tổng và số tham số được học. Vì vậy, số tham số trainable chỉ phản ánh một phần chi phí; accuracy, thời gian và bộ nhớ bổ sung những mặt khác nhau của phép so sánh.

4.3.4 Khoảng cách train/validation và lỗi theo lớp

Cuối lịch 20 epoch, cả hai bản full đều đạt accuracy train gần 99%, trong khi validation thấp hơn rõ rệt. ResNet18 có khoảng cách lớn hơn Swin-T. Đường cong cho thấy mô hình tiếp tục khớp dữ liệu train nhưng khả năng phân loại ảnh validation tăng ít hơn, phù hợp với overfitting. Chỉ số train được đo trên ảnh augmentation trong lúc cập nhật, còn validation được đo sau epoch ở chế độ eval; vì vậy, khoảng cách này dùng để đọc xu hướng học. Việc chọn checkpoint theo validation giúp lấy mô hình ở thời điểm phân loại tốt nhất trên tập này, thay vì mặc nhiên lấy epoch cuối.

Lỗi còn tập trung ở các lớp gần nhau về ngoại hình. Khi gộp ba seed, các lớp có recall thấp nhất của ResNet18 full gồm `boy`, `girl` và `otter`; với Swin-T full là `girl`, `boy` và `oak_tree`. Bảng sau trình bày các giá trị này. Mỗi lớp có 300 dự đoán, gồm 100 ảnh test được đánh giá bởi từng seed.

ResNet18 full		Swin-T full	
Lớp	Recall (%)	Lớp	Recall (%)
`boy`	50,0	`girl`	61,7
`girl`	53,0	`boy`	63,3
`otter`	57,0	`oak_tree`	67,3

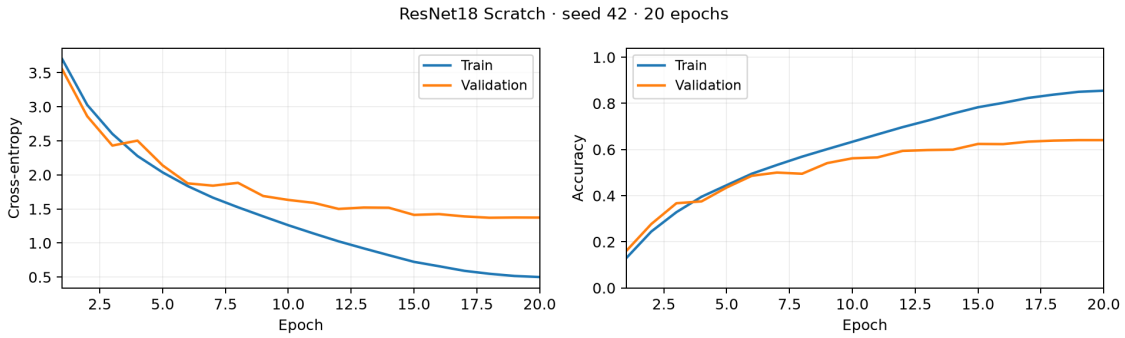
Bảng 4.3: Ba lớp có recall thấp nhất của mỗi bản full fine-tuning, tính từ confusion matrix test gộp seed 42, 43 và 44; support 300 dự đoán mỗi lớp.

Ở cả hai mô hình, cặp nhầm phổ biến nhất là `oak_tree` thành `maple_tree`. Swin-T mắc

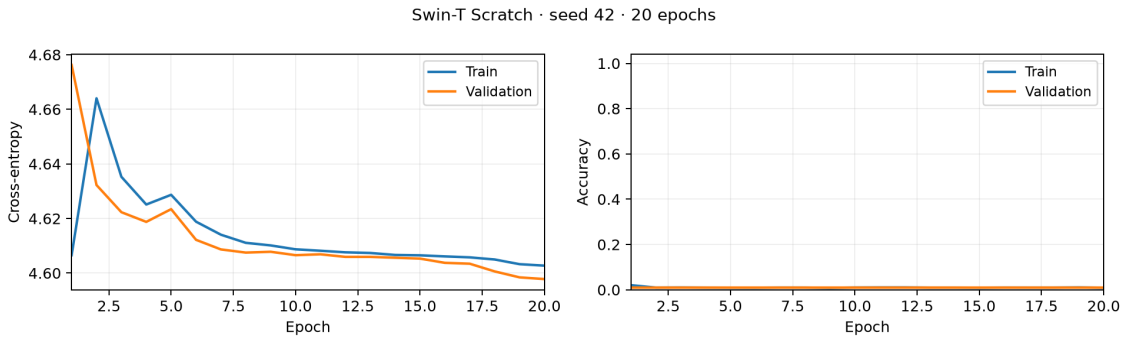
71 lỗi loại này, nhiều hơn 56 lỗi của ResNet18, dù accuracy toàn bộ cao hơn. Kết quả cho thấy một mô hình tốt hơn về điểm tổng thể vẫn có thể yếu hơn trên một cặp lớp cụ thể. Các ảnh sai có confidence cao trong gallery minh họa sự giống nhau về hình dáng và kết cấu ở độ phân giải 32×32; chúng được chọn để xem lỗi khó, không phải để ước lượng tỷ lệ từng loại lỗi.

4.4 Đường cong và lỗi phân loại

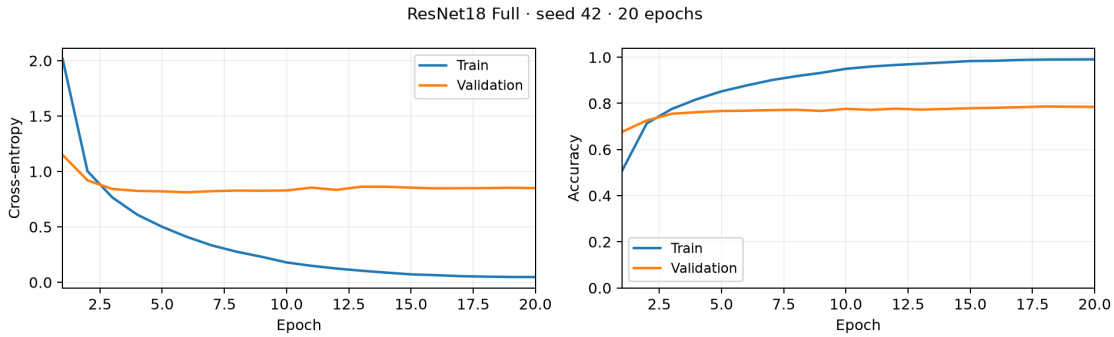
Các hình dùng seed 42 đã định trước, không chọn seed theo kết quả test. Confusion matrix dùng chỉ số lớp 0–99; danh sách tên lớp và số đếm đầy đủ nằm trong các tệp CSV kèm báo cáo.



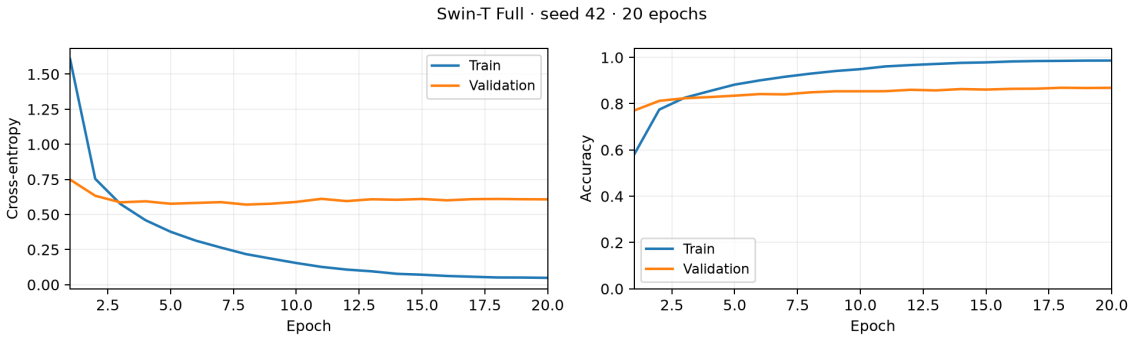
Hình 4.1: Cross-entropy và accuracy trên train/validation của ResNet18 scratch ở seed 42.



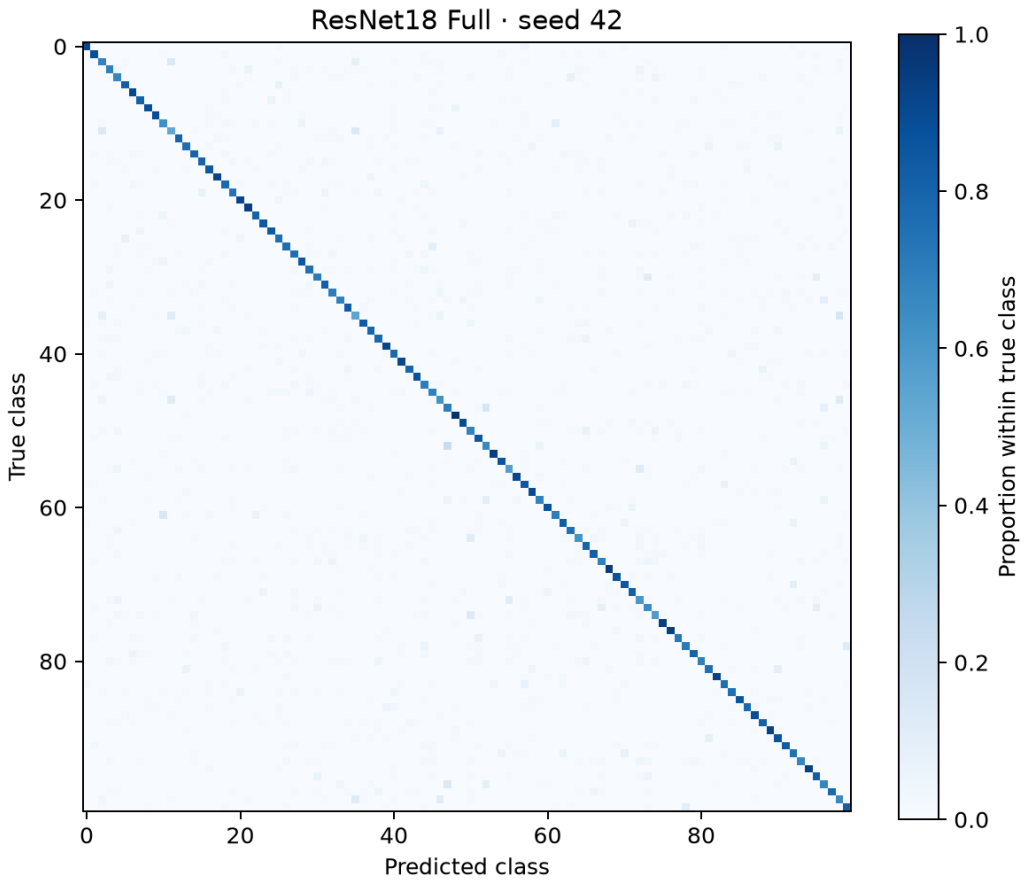
Hình 4.2: Cross-entropy và accuracy trên train/validation của Swin-T scratch ở seed 42.



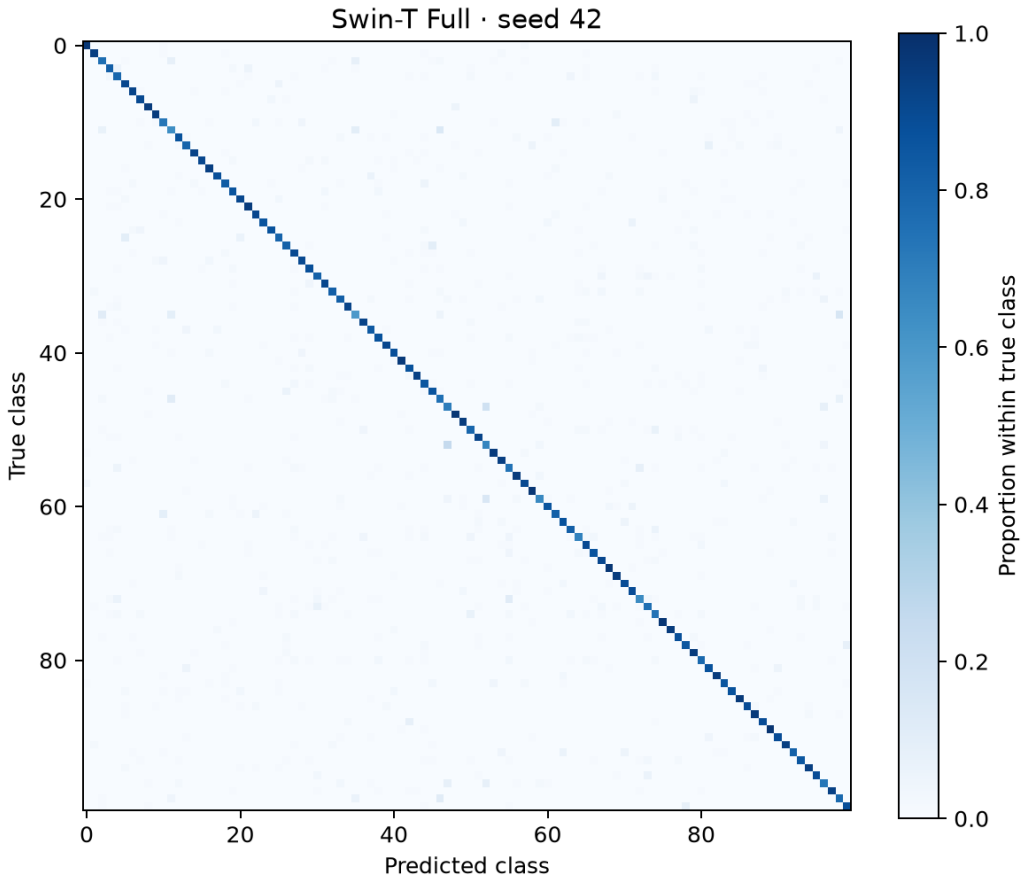
Hình 4.3: Cross-entropy và accuracy trên train/validation của ResNet18 pretrained full fine-tuning ở seed 42.



Hình 4.4: Cross-entropy và accuracy trên train/validation của Swin-T pretrained full fine-tuning ở seed 42.



Hình 4.5: Confusion matrix test của ResNet18 full ở seed 42; hàng là nhãn thực tế.



Hình 4.6: Confusion matrix test của Swin-T full ở seed 42.



Hình 4.7: Các dự đoán sai có confidence cao của ResNet18 full ở seed 42, hiển thị ảnh gốc 32×32.



Hình 4.8: Các dự đoán sai có confidence cao của Swin-T full ở seed 42.

Chương 5

Kết quả đối chứng khởi tạo Swin-T

Cấu hình	Validation (%)	Test (%)	Macro-F1
Swin-T Scratch	60,23 ± 0,28	60,00 ± 0,47	0,5990 ± 0,0049
Swin-T Full	86,13 ± 0,12	86,45 ± 0,19	0,8644 ± 0,0020

Bảng 5.1: Accuracy validation/test và macro-F1 test của Swin-T scratch và pretrained full fine-tuning dưới cùng lịch 40 epoch, trung bình và SD mẫu qua seed 42, 43 và 44.

Với lịch 40 epoch, Swin-T scratch đạt accuracy test $60{,}00 \pm 0{,}47\%$, cao hơn rõ rệt so với 2,19% của lịch 20 epoch. Đường cong train và validation đều cải thiện, cho thấy mô hình đã học được cách phân biệt các lớp. Tuy vậy, accuracy cuối lịch trên train gần 90%, còn validation khoảng 60%: khớp dữ liệu train vẫn dễ hơn phân loại ảnh mới. Các checkpoint scratch tốt nhất được chọn trong khoảng epoch 36–37.

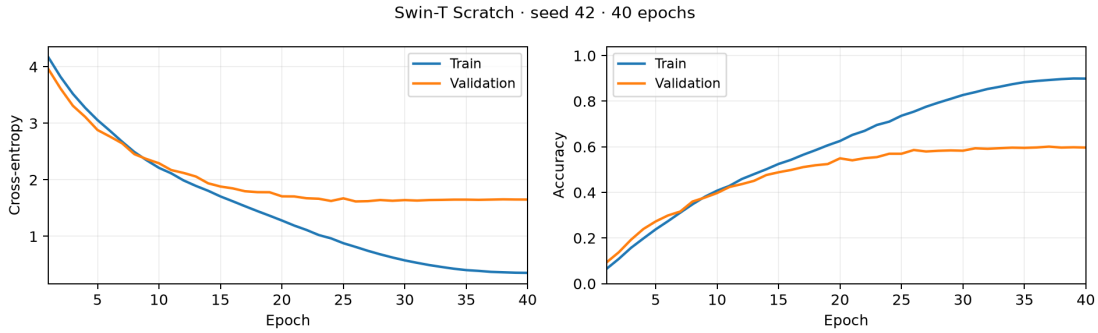
Swin-T pretrained đạt $86{,}45 \pm 0{,}19\%$ dưới cùng lịch. Để so sánh, lấy accuracy của bản pretrained trừ bản scratch ở từng seed, rồi tính trung bình ba chênh lệch. Kết quả là $26{,}45 \pm 0{,}40$ điểm phần trăm; bản pretrained cao hơn ở cả ba cặp. Như vậy, ngay khi bản scratch đã học được, khởi tạo từ ImageNet vẫn đem lại lợi ích lớn. Hai bản có cùng kiến trúc, cập nhật toàn bộ tham số và dùng cùng lịch, nên phép so sánh này rõ hơn về vai trò của khởi tạo so với lịch 20 epoch.

Kiến trúc	Chế độ	Tổng (M)	Học (M)	Giây	MiB
Swin-T	Scratch	27,596	27,596	428,1 ± 1,6	2453,2 ± 3,7
Swin-T	Full	27,596	27,596	429,7 ± 0,8	2456,7 ± 2,2

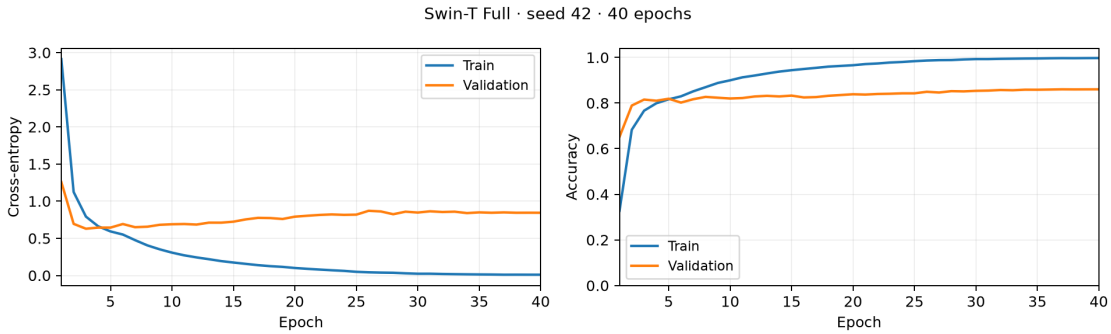
Bảng 5.2: Chi phí lịch 40 epoch; thời gian train và peak allocated memory là trung bình và SD mẫu qua ba seed.

Chi phí hai bản gần nhau: thời gian train đều khoảng 429 giây và bộ nhớ cấp phát cực đại khoảng 2455 MiB. Khác biệt chính trong đối chứng này nằm ở chất lượng phân loại, không phải số tham số hoặc ngân sách tính toán.

So với lịch 20 epoch, lịch mới giúp bản scratch học tốt hơn rõ rệt, còn bản pretrained vẫn đạt khoảng 86,5%. Học lâu hơn vì vậy không tự bảo đảm tăng accuracy. Tuy nhiên, lịch mới thay đổi đồng thời số epoch, learning rate, warmup và weight decay; mức cải thiện của scratch thuộc về cả thiết lập mới, không thể quy riêng cho việc tăng từ 20 lên 40 epoch.



Hình 5.1: Cross-entropy và accuracy trên train/validation của Swin-T scratch trong lịch 40 epoch ở seed 42.



Hình 5.2: Cross-entropy và accuracy trên train/validation của Swin-T pretrained full fine-tuning trong lịch 40 epoch ở seed 42.

Chương 6

Kết luận và giới hạn

6.1 Kết luận

Fine-tuning giúp cả ResNet18 và Swin-T thích nghi với CIFAR-100 tốt hơn so với chỉ học head trên backbone đóng băng. Full fine-tuning đạt accuracy cao nhất ở mỗi kiến trúc. Swin-T full chính xác hơn, còn ResNet18 full có ưu thế về thời gian và bộ nhớ.

Kết quả Swin-T cho thấy cần xem cả quá trình học trước khi giải thích khoảng cách scratch và pretrained. Ở lịch 20 epoch, scratch gần mức đoán ngẫu nhiên trong một số lượt. Với lịch chung 40 epoch, scratch đã đạt 60,00% nhưng vẫn thấp hơn rõ rệt so với 86,45% của bản pretrained. Do đó, lợi ích của khởi tạo ImageNet vẫn hiện diện khi bản scratch được huấn luyện hiệu quả hơn.

6.2 Giới hạn

Kết quả được đo trên CIFAR-100 với ba seed và một môi trường phần cứng. Ảnh gốc chỉ có kích thước 32×32; resize lên 96×96 không bổ sung chi tiết thị giác. ResNet18 và Swin-T khác cả số tham số lẫn cách học biểu diễn, nên kết quả so sánh hai mô hình này không đại diện cho mọi CNN và Transformer.

Trong lịch 20 epoch, scratch và partial/full dùng learning rate khác nhau; vì vậy, đây là so sánh các thiết lập huấn luyện cụ thể. Đối chứng chung 40 epoch chỉ dành cho Swin-T và chưa khảo sát ngân sách dài hơn. Phạm vi hỗ trợ của AI được trình bày trong `AI_USE_REPORT.md`.

Tài liệu tham khảo

[1] Lê Thành Sách, Nguyễn Quốc Minh. *Bài tập lớn A1: CNN vs Transformer trên tập ảnh lớn*. CO5085, học kỳ 261.

[2] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. 2009. Trang dataset: `https://www.cs.toronto.edu/~kriz/cifar.html`.

[3] uoft-cs. *CIFAR100 dataset card*; license: unknown. `https://huggingface.co/datasets/uoft-cs/cifar100`.

[4] K. He và cộng sự. *Deep Residual Learning for Image Recognition*. CVPR 2016. `https://arxiv.org/abs/1512.03385`.

[5] Z. Liu và cộng sự. *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. ICCV 2021. `https://arxiv.org/abs/2103.14030`.

[6] Torchvision. ResNet18 và Swin-T model API; sử dụng weight enum IMAGENET1K_V1. `https://docs.pytorch.org/vision/stable/models.html`.